

JAVA

Competitive Programming — Complete Reference

*Helper methods for Arrays · Strings · HashMap · Stack · Queue · PriorityQueue · Bit tricks —
and everything in between*

Prepared for TCS NQT & general competitive-programming use

Table of Contents

Table of Contents	2
1. Arrays (java.util.Arrays)	1
Reverse-sort an int[] (no direct method exists):	1
2. ArrayList (java.util.ArrayList)	2
3. String & StringBuilder	3
4. HashMap / TreeMap / LinkedHashMap	4
Frequency counter idiom:	4
5. HashSet / TreeSet	5
6. Stack (use ArrayDeque, not legacy Stack)	5
Declaration:	5
7. Queue / Deque (java.util.ArrayDeque)	6
8. PriorityQueue (Min-Heap by default)	6
9. Math & Integer/Long utilities	7
GCD / LCM (write manually):	7
10. Character utilities	8
11. Collections utility class	8
12. Fast I/O (critical for timed judges like TCS iON)	8
BufferedReader + StringTokenizer (fast, preferred for large input):	8
13. Bit Manipulation (operators, not methods)	9
14. Quick decision table — which structure to use	9

1. Arrays (java.util.Arrays)

Method	Use
<code>Arrays.sort(arr)</code>	Sorts the array ascending, in place. Example: <code>[3,1,2] → [1,2,3]</code> . $O(n \log n)$.
<code>Arrays.sort(arr, from, to)</code>	Sorts only the sub-range <code>[from, to)</code> (<code>to</code> is exclusive), leaving the rest untouched.
<code>Arrays.sort(arr, cmp)</code>	Sorts using a custom Comparator. Only works on <code>Object[]</code> (e.g. <code>Integer[]</code>) or 2D arrays — NOT plain <code>int[]/double[]</code> , since primitives have no Comparator overload.
<code>Arrays.fill(arr, val)</code>	Overwrites every element with <code>val</code> . Example: <code>Arrays.fill(arr, -1) →</code> all <code>-1</code> s.
<code>Arrays.fill(arr, from, to, val)</code>	Overwrites only indices <code>[from, to)</code> with <code>val</code> ; rest of the array is untouched.
<code>Arrays.copyOf(arr, newLen)</code>	Returns a new array of length <code>newLen</code> , copying old values in and padding extra slots with <code>0/null</code> .
<code>Arrays.copyOfRange(arr, from, to)</code>	Returns a new array containing just the sub-range <code>[from, to)</code> of the original.
<code>Arrays.equals(a, b)</code>	Checks two 1D arrays have the same length and equal elements at every index (unlike <code>a==b</code> , which just compares references).
<code>Arrays.deepEquals(a, b)</code>	Like <code>Arrays.equals</code> but recurses into nested arrays — use for 2D/3D arrays.
<code>Arrays.toString(arr)</code>	Builds a readable String like <code>"[1, 2, 3]"</code> for a 1D array — use for printing/debugging.
<code>Arrays.deepToString(arr)</code>	Like <code>Arrays.toString</code> but recurses into nested arrays — use for printing 2D/3D arrays.
<code>Arrays.binarySearch(arr, key)</code>	Finds key's index in an ALREADY-SORTED array in $O(\log n)$. Returns a negative number if key is absent — sort first or results are undefined.
<code>Arrays.asList(arr)</code>	Wraps an <code>Object[]</code> (e.g. <code>Integer[]</code> , <code>String[]</code>) as a fixed-size List view backed by the same array — you can <code>set()</code> elements but NOT <code>add()/remove()</code> . Does not work directly on <code>int[]</code> .
<code>Arrays.stream(arr).sum()/max()/min()</code>	Converts an <code>int[]</code> to an <code>IntStream</code> to quickly get its sum, max, or min without writing a loop.

Reverse-sort an `int[]` (no direct method exists):

```
Integer[] boxed = Arrays.stream(arr).boxed().toArray(Integer[]::new);
Arrays.sort(boxed, Collections.reverseOrder());
```

💡 `int[]` cannot be sorted in reverse directly — box it to `Integer[]` first.

2. ArrayList (java.util.ArrayList)

Method	Use
<code>add(x)</code>	Appends <code>x</code> to the end of the list. $O(1)$ on average (amortized).
<code>add(index, x)</code>	Inserts <code>x</code> at position <code>index</code> , shifting later elements right by one. $O(n)$ because of the shift.
<code>get(index)</code>	Returns the element at position <code>index</code> . $O(1)$ — ArrayList is backed by an array.
<code>set(index, x)</code>	Replaces the element at position <code>index</code> with <code>x</code> and returns the old value.
<code>remove(index)</code>	Removes the element at position <code>index</code> (by position!) and shifts later elements left. $O(n)$.
<code>remove(Object o)</code>	Removes the first element equal to <code>o</code> (by value!), not by index. Tip: for Integer lists, <code>remove((Integer) 5)</code> removes the value 5, while <code>remove(5)</code> removes the element at index 5.
<code>size()</code>	Returns how many elements are currently in the list (note: <code>size()</code> , not <code>length</code> like arrays).
<code>contains(x)</code>	Checks whether <code>x</code> is present anywhere in the list by scanning it. $O(n)$.
<code>indexOf(x)</code>	Returns the index of the first occurrence of <code>x</code> , or <code>-1</code> if it isn't in the list.
<code>isEmpty()</code>	Returns true if the list has zero elements — shorthand for <code>size() == 0</code> .
<code>clear()</code>	Removes every element, leaving the list empty (<code>size()</code> becomes 0).
<code>Collections.sort(list)</code>	Sorts ascending using natural order. Example: <code>Collections.sort(list)</code> turns <code>[5,2,8]</code> into <code>[2,5,8]</code> . $O(n \log n)$.
<code>Collections.sort(list, cmp)</code>	Sorts using a custom Comparator you provide. Example: <code>Collections.sort(list, (a, b) -> b - a)</code> sorts descending.
<code>Collections.reverse(list)</code>	Reverse in place
<code>Collections.max/min(list)</code>	Extremes
<code>Collections.frequency(list, x)</code>	Count occurrences
<code>list.toArray(new Integer[0])</code>	Copies the list's elements into a new <code>Integer[]</code> array of the right size.

3. String & StringBuilder

String is immutable — every modification creates a new object. Use StringBuilder inside loops.

Method	Use
<code>s.length()</code>	Returns the number of characters in the string.
<code>s.charAt(i)</code>	Returns the single character at index <code>i</code> (0-based). Throws an exception if <code>i</code> is out of range.
<code>s.substring(a,b)</code>	Returns the characters from index <code>a</code> up to (but not including) index <code>b</code> .
<code>s.indexOf(ch/str)</code>	Returns the index of the first occurrence of the character/substring, or <code>-1</code> if not found.
<code>s.lastIndexOf(ch/str)</code>	Like <code>indexOf</code> but searches from the end, returning the last occurrence's index.
<code>s.split(regex)</code>	Splits the string on the given regex pattern and returns the pieces as a <code>String[]</code> .
<code>s.toCharArray()</code>	Copies the string's characters into a new <code>char[]</code> so you can index/modify them directly.
<code>s.equals(t)</code> / <code>s.equalsIgnoreCase(t)</code>	Compares actual text content (<code>equalsIgnoreCase</code> ignores case). Always use this instead of <code>==</code> for strings, since <code>==</code> compares object references, not content.
<code>s.compareTo(t)</code>	Compares two strings alphabetically: negative if <code>s < t</code> , 0 if equal, positive if <code>s > t</code> .
<code>s.trim()</code> / <code>s.strip()</code>	Removes leading/trailing whitespace. <code>strip()</code> (Java 11+) also handles Unicode whitespace that <code>trim()</code> misses.
<code>s.toUpperCase()</code> / <code>s.toLowerCase()</code>	Returns a new string with all letters converted to upper/lower case (original string is unchanged, since Strings are immutable).
<code>s.replace(old,new)</code>	Returns a new string with every occurrence of <code>old</code> replaced by <code>new</code> .
<code>s.contains(sub)</code>	Returns true if <code>sub</code> appears anywhere inside the string.
<code>s.startsWith()</code> / <code>s.endsWith()</code>	Checks whether the string begins or ends with the given prefix/suffix.
<code>String.valueOf(x)</code>	Converts almost any type (<code>int</code> , <code>char</code> , <code>boolean</code> , <code>object</code> , etc.) into its String representation.
<code>String.join(",", list)</code>	Joins a list/array of strings into one string, separated by the given delimiter — e.g. joins <code>["a","b"]</code> into <code>"a,b"</code> .
<code>sb.append(x)</code>	Adds <code>x</code> to the end of the StringBuilder's buffer in $O(1)$ amortized time — the efficient alternative to <code>s = s + x</code> .
<code>sb.insert(i,x)</code>	Inserts <code>x</code> into the buffer at position <code>i</code> , shifting later characters right.
<code>sb.deleteCharAt(i)</code>	Removes the single character at position <code>i</code> from the buffer.
<code>sb.reverse()</code>	Reverses the buffer's characters in place.
<code>sb.setCharAt(i,ch)</code>	Overwrites the character at position <code>i</code> with <code>ch</code> .

Method	Use
<code>sb.toString()</code>	Converts the buffer's contents into an immutable String (call this when you're done building).

💡 Never do `s = s + x` inside a loop ($O(n^2)$ total). Always use `StringBuilder`.

4. HashMap / TreeMap / LinkedHashMap

Method	Use
<code>map.put(k,v)</code>	Inserts key <code>k</code> with value <code>v</code> , or overwrites <code>v</code> if <code>k</code> is already present.
<code>map.get(k)</code>	Returns the value mapped to <code>k</code> , or null if <code>k</code> isn't in the map.
<code>map.getDefault(k,def)</code>	Like <code>get(k)</code> , but returns <code>def</code> instead of null when <code>k</code> is missing — avoids a manual null check.
<code>map.containsKey(k)</code>	Returns true if the map has an entry for key <code>k</code> .
<code>map.remove(k)</code>	Deletes the entry for key <code>k</code> (if present) and returns its old value.
<code>map.putIfAbsent(k,v)</code>	Inserts <code>k→v</code> only if <code>k</code> isn't already in the map; otherwise leaves the existing value untouched.
<code>map.merge(k,v,(o,n)->o+n)</code>	If <code>k</code> is absent, inserts <code>k→v</code> . If <code>k</code> is present, combines the old value (<code>o</code>) and new value (<code>n</code>) with the given function — the standard idiom for building a frequency map.
<code>map.compute(k,(key,val)->val+1)</code>	Recomputes the value for key using a function you supply, based on the current value (<code>val</code> may be null if absent).
<code>map.keySet() / .values() / .entrySet()</code>	Returns a view for iterating just the keys, just the values, or key-value pairs (<code>Map.Entry</code>) together.
<code>map.size()</code>	Returns the number of key-value entries in the map.
TreeMap: <code>firstKey()/lastKey()</code>	Returns the smallest/largest key currently in the <code>TreeMap</code> (keys are kept sorted automatically).
TreeMap: <code>ceilingKey(k)/floorKey(k)</code>	Returns the smallest key $\geq k$ (ceiling) or largest key $\leq k$ (floor), or null if none exists.
TreeMap: <code>higherKey(k)/lowerKey(k)</code>	Like ceiling/floor but strict: smallest key $> k$, or largest key $< k$.
TreeMap: <code>pollFirstEntry()/pollLastEntry()</code>	Removes and returns the entry with the smallest/largest key.

Frequency counter idiom:

```
map.merge(x, 1, Integer::sum); // increment count of x
```

5. HashSet / TreeSet

Method	Use
<code>set.add(x)</code>	Inserts <code>x</code> if it's not already present; does nothing (no error) if it's a duplicate.
<code>set.contains(x)</code>	Checks membership in average $O(1)$ time (HashSet) — much faster than scanning a list.
<code>set.remove(x)</code>	Removes <code>x</code> from the set if present.
<code>set.retainAll(other)</code>	Keeps only the elements also present in <code>other</code> — computes the set intersection, in place.
<code>set.removeAll(other)</code>	Removes every element that's also in <code>other</code> — computes the set difference, in place.
<code>set.addAll(other)</code>	Adds every element of <code>other</code> into this set — computes the set union, in place.
TreeSet: <code>first()/last()</code>	Returns the smallest/largest element (TreeSet keeps elements sorted automatically).
TreeSet: <code>ceiling(x)/floor(x)</code>	Returns the smallest element $\geq x$ (ceiling) or largest element $\leq x$ (floor).
TreeSet: <code>higher(x)/lower(x)</code>	Like ceiling/floor but strict: smallest element $> x$, or largest element $< x$.
TreeSet: <code>pollFirst()/pollLast()</code>	Removes and returns the smallest/largest element.

6. Stack (use ArrayDeque, not legacy Stack)

Method	Use
<code>stack.push(x)</code>	Adds <code>x</code> to the top of the stack (LIFO — last in, first out).
<code>stack.pop()</code>	Removes and returns the top element. Throws if the stack is empty.
<code>stack.peek()</code>	Returns the top element without removing it.
<code>stack.isEmpty()</code>	Returns true if the stack has no elements.

Declaration:

```
Deque<Integer> stack = new ArrayDeque<>();
```

7. Queue / Deque (java.util.ArrayDeque)

Deque can act as Queue, Stack, or double-ended queue. Prefer ArrayDeque over LinkedList.

Method	Use
<code>dq.offer(x) / offerLast(x)</code>	Adds <code>x</code> to the back of the deque — the standard way to enqueue for FIFO use.
<code>dq.offerFirst(x)</code>	Adds <code>x</code> to the front of the deque.
<code>dq.poll() / pollFirst()</code>	Removes and returns the front element, or null (not an exception) if the deque is empty.
<code>dq.pollLast()</code>	Removes and returns the back element, or null if empty.
<code>dq.peek() / peekFirst()</code>	Returns the front element without removing it (null if empty).
<code>dq.peekLast()</code>	Returns the back element without removing it (null if empty).
<code>dq.isEmpty() / size()</code>	Checks whether the deque is empty, or returns how many elements it holds.

💡 Sliding window maximum: keep a `Deque<Integer>` of indices, values monotonic decreasing.

8. PriorityQueue (Min-Heap by default)

Method	Use
<code>new PriorityQueue<>()</code>	Creates a min-heap: <code>poll()</code> always returns the smallest element first, using natural ordering.
<code>new PriorityQueue<>(Collections.reverseOrder())</code>	Creates a max-heap: <code>poll()</code> always returns the largest element first.
<code>new PriorityQueue<>((a,b)->a[0]-b[0])</code>	Creates a heap with a custom ordering rule — here, e.g. <code>int[]</code> pairs are ordered by their first element ascending.
<code>pq.offer(x) / add(x)</code>	Inserts <code>x</code> into the heap, maintaining heap order. $O(\log n)$.
<code>pq.poll()</code>	Removes and returns the top (min or max, depending on setup) element. $O(\log n)$.
<code>pq.peek()</code>	Returns the top element without removing it. $O(1)$.
<code>pq.isEmpty() / size()</code>	Checks whether the heap is empty, or returns how many elements it holds.

💡 Iterating a `PriorityQueue` does NOT give sorted order — only repeated `poll()` does.

9. Math & Integer/Long utilities

Method	Use
<code>Math.max(a,b) / min(a,b)</code>	Returns the larger/smaller of two values.
<code>Math.abs(x)</code>	Returns the absolute (non-negative) value of <code>x</code> .
<code>Math.pow(base,exp)</code>	Returns base raised to the power <code>exp</code> , as a double (not an int).
<code>Math.sqrt(x)</code>	Returns the square root of <code>x</code> , as a double.
<code>Math.ceil(x) / floor(x)</code>	Rounds <code>x</code> up (ceil) or down (floor) to the nearest whole number, returned as a double.
<code>Math.round(x)</code>	Rounds <code>x</code> to the nearest whole number (0.5 rounds up).
<code>Math.log(x) / log10(x)</code>	Returns the natural logarithm (base <code>e</code>) or base-10 logarithm of <code>x</code> .
<code>Integer.MAX_VALUE / MIN_VALUE</code>	The largest/smallest value an int can hold: 2147483647 / -2147483648. Adding 1 past <code>MAX_VALUE</code> silently wraps to <code>MIN_VALUE</code> (overflow).
<code>Long.MAX_VALUE / MIN_VALUE</code>	Like Integer's constants but for long, which has a much larger range — use when sums/products might exceed ~2.1 billion.
<code>Integer.parseInt(str)</code>	Converts a numeric String into an int. Throws <code>NumberFormatException</code> if <code>str</code> isn't a valid integer.
<code>Integer.toBinaryString(x)</code>	Returns <code>x</code> 's binary representation as a String (no leading zeros), e.g. 5 → "101".
<code>Integer.bitCount(x)</code>	Returns how many bits are set to 1 in <code>x</code> (a.k.a. popcount).
<code>Integer.numberOfTrailingZeros(x)</code>	Returns the position (0-indexed from the right) of the lowest set bit.
<code>Integer.compare(a,b)</code>	Returns -1, 0, or 1 depending on whether <code>a < b</code> , <code>a == b</code> , or <code>a > b</code> — safer than <code>a-b</code> , which can overflow for extreme values.

GCD / LCM (write manually):

```
static int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }  
static int lcm(int a, int b) { return a / gcd(a, b) * b; }
```

💡 Use long for sums/products that could exceed ~2.1 billion — int overflows silently.

10. Character utilities

Method	Use
<code>Character.isDigit(ch)</code>	Returns true if <code>ch</code> is a digit character ('0' through '9').
<code>Character.isLetter(ch)</code>	Returns true if <code>ch</code> is an alphabetic letter.
<code>Character.isLetterOrDigit(ch)</code>	Returns true if <code>ch</code> is either a letter or a digit.
<code>Character.isUpperCase/isLowerCase(ch)</code>	Checks whether <code>ch</code> is uppercase or lowercase.
<code>Character.toUpperCase/toLowerCase(ch)</code>	Returns the upper/lower-case version of <code>ch</code> .
<code>Character.getNumericValue(ch)</code>	Converts a digit character to its int value, e.g. '7' → 7 (useful when parsing digits from a string one char at a time).

11. Collections utility class

Method	Use
<code>Collections.sort(list)</code>	Sort ascending
<code>Collections.reverse(list)</code>	Reverse order
<code>Collections.shuffle(list)</code>	Randomly reorders the list's elements in place (like shuffling a deck of cards).
<code>Collections.max/min(list)</code>	Extremes
<code>Collections.swap(list,i,j)</code>	Swaps the elements at positions <code>i</code> and <code>j</code> in place.
<code>Collections.unmodifiableList(list)</code>	Returns a read-only wrapper around <code>list</code> — attempts to modify it throw <code>UnsupportedOperationException</code> . The underlying list can still change if modified through the original reference.
<code>Collections.nCopies(n,x)</code>	Returns an immutable list containing <code>x</code> repeated <code>n</code> times, e.g. <code>nCopies(3, 0) → [0, 0, 0]</code> .

12. Fast I/O (critical for timed judges like TCS iON)

BufferedReader + StringTokenizer (fast, preferred for large input):

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
int n = Integer.parseInt(br.readLine().trim());
StringTokenizer st = new StringTokenizer(br.readLine());
int a = Integer.parseInt(st.nextToken());
int b = Integer.parseInt(st.nextToken());
```

💡 *Scanner is simpler but noticeably slower on large inputs — prefer BufferedReader under strict time limits.*

13. Bit Manipulation (operators, not methods)

Method	Use
$x \& 1$	Checks the lowest bit: result is 1 if x is odd, 0 if x is even.
$x \gg 1$	Shifts bits right by one position — equivalent to integer division by 2.
$x \ll 1$	Shifts bits left by one position — equivalent to multiplying by 2.
$x \& (x-1)$	Clears the lowest set (1) bit of x, leaving the rest unchanged.
$x (1 \ll i)$	Sets bit i to 1, leaving all other bits unchanged.
$x \& \sim(1 \ll i)$	Clears bit i to 0, leaving all other bits unchanged.
$x \wedge (1 \ll i)$	Flips (toggles) bit i: 0 becomes 1, 1 becomes 0.
$(x \gg i) \& 1$	Checks whether bit i is set: returns 1 if set, 0 if not.
$x \& -x$	Isolates the lowest set bit of x, zeroing out all other bits (relies on two's-complement representation).

14. Quick decision table — which structure to use

Method	Use
Fast lookup by key	HashMap — $O(1)$ average get/put by key, no ordering guarantee.
Sorted keys, range queries	TreeMap — keeps keys sorted, supports floor/ceiling/range operations in $O(\log n)$.
Unique elements, fast membership	HashSet — $O(1)$ average contains(), no ordering guarantee.
Unique + sorted	TreeSet — like HashSet but keeps elements sorted, $O(\log n)$ operations.
LIFO (undo, parentheses, DFS)	Deque used as a Stack via push()/pop()/peek() — last in, first out.
FIFO (BFS, scheduling)	Deque used as a Queue via offer()/poll()/peek() — first in, first out.
Repeatedly get min/max	PriorityQueue — $O(\log n)$ insert/remove while always giving you the current min or max.
Index-based fast access	ArrayList / array — $O(1)$ get(index)/set(index) by position.
Insert/delete at both ends	ArrayDeque — $O(1)$ add/remove at both the front and back.